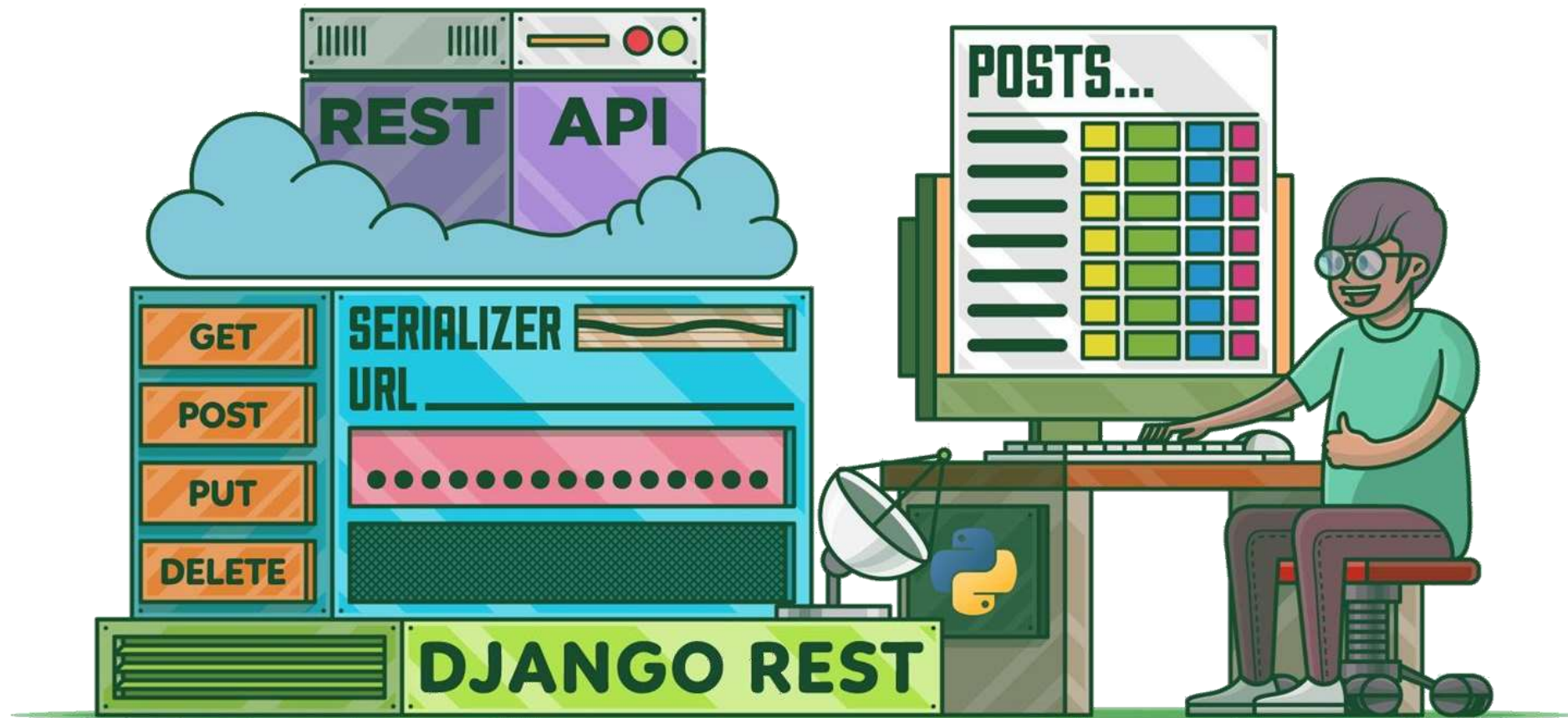
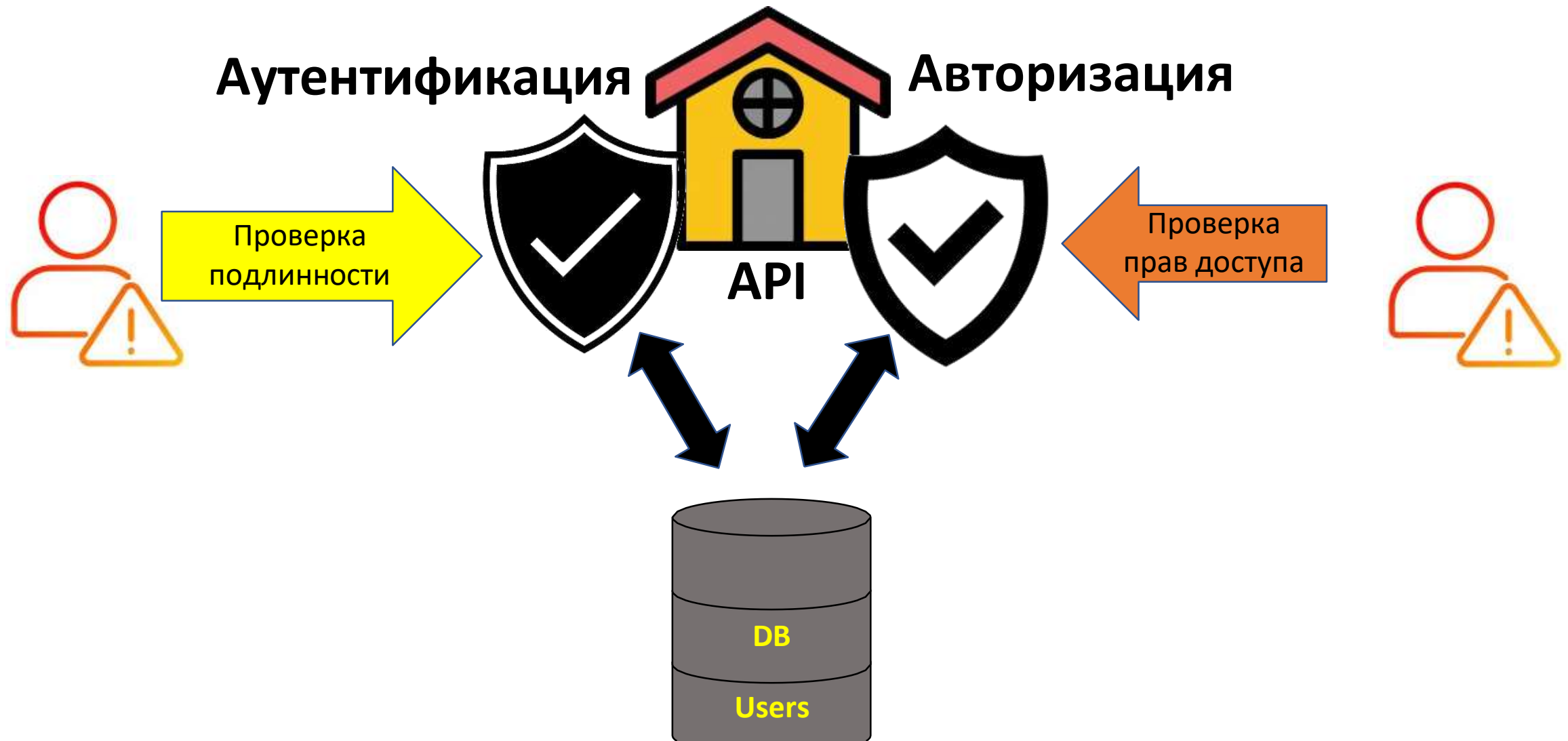




Django REST: авторизация и аутентификация

Авторизация и аутентификация в защите API



Схемы авторизации

Схемы авторизации = наборы классов, позволяющие реализовать доступ к данным в Django.

1. TokenAuthentication
2. SessionAuthentication
3. BasicAuthentication
4. JWTAuthentication
5. OAuth2Authentication

TokenAuthentication

TokenAuthentication — это простая схема аутентификации, предоставляемая DRF из коробки. При этой схеме каждому пользователю присваивается уникальный токен, который используется для идентификации при каждом запросе к API.

Как это работает:

- При успешной аутентификации (например, через форму входа) пользователю выдается токен.
- Клиент отправляет этот токен в заголовке каждого последующего запроса к API.
- Сервер проверяет токен и, если он действителен, предоставляет доступ к запрошенным ресурсам.

Преимущества:

- Простота внедрения и использования.
- Хорошо подходит для мобильных приложений и сервисов, где сессии неудобны.
- Токены можно легко отозвать или обновить при необходимости.

Недостатки:

- Токены хранятся на сервере, что может создавать нагрузку при большом количестве пользователей.
- Отсутствие встроенного механизма обновления токенов (в отличие от JWT).
- При утечке токена злоумышленник получает полный доступ до его отзыва.

SessionAuthentication

SessionAuthentication использует стандартный механизм сессий Django для аутентификации пользователей. Это означает, что аутентификация происходит с использованием cookie, которые устанавливаются при входе пользователя.

Как это работает:

- Пользователь проходит аутентификацию (например, через форму входа), и сервер устанавливает сессионную cookie.
- При последующих запросах браузер автоматически отправляет эту cookie, позволяя серверу идентифицировать пользователя.
- DRF использует информацию из сессии для определения прав доступа пользователя.

Преимущества:

- Глубокая интеграция с стандартной системой аутентификации Django.
- Поддерживает механизмы защиты от CSRF-атак.
- Хорошо подходит для веб-приложений, где используется браузер.

Недостатки:

- Неудобна для использования в мобильных приложениях или сторонних сервисах, которые не работают с cookie.

BasicAuthentication

BasicAuthentication реализует базовую HTTP-аутентификацию, где учетные данные (имя пользователя и пароль) передаются в каждом запросе в закодированном виде.

Как это работает:

- Клиент отправляет имя пользователя и пароль в заголовке Authorization каждого запроса.
- Сервер декодирует и проверяет эти данные при каждом запросе.

Преимущества:

- Очень простая в реализации и использовании.
- Поддерживается практически всеми HTTP-клиентами из коробки.

Недостатки:

- Не безопасна при использовании без HTTPS, так как данные передаются в закодированном, но не зашифрованном виде.
- Каждый запрос содержит учетные данные, что увеличивает риск их перехвата.
- Неудобно управлять сессиями и отзывать доступ без изменения пароля.

JWTAuthentication

JWTAuthentication (JSON Web Token Authentication) использует токены в формате JWT для аутентификации. JWT — это компактный и самодостаточный токен, который содержит зашифрованную информацию о пользователе и его правах доступа.

Как это работает:

- Пользователь аутентифицируется и получает JWT-токен, содержащий закодированную информацию и подпись.
- Клиент отправляет этот токен в заголовке Authorization каждого запроса.
- Сервер проверяет подпись токена и извлекает из него информацию о пользователе без необходимости обращения к базе данных.

Преимущества:

- **Статусность:** сервер не хранит информацию о сессиях, что облегчает масштабирование.
- **Компактность и самодостаточность** токена.
- **Возможность установки времени жизни токена и использования механизма обновления (refresh tokens).**
- **Широко используется и поддерживается многими библиотеками.**

Недостатки:

- **При компрометации токена злоумышленник имеет доступ до истечения срока действия токена.**
- **Отзыв токена до истечения его срока действия может быть сложным без дополнительной логики.**
- **Требует тщательной настройки для обеспечения безопасности (например, правильная обработка обновления токенов).**

OAuth2Authentication

OAuth2Authentication реализует протокол OAuth 2.0, который предоставляет механизм авторизации, позволяющий приложениям получать ограниченный доступ к ресурсам пользователя на стороннем сервисе без передачи его учетных данных.

Как это работает:

- Пользователь предоставляет приложению разрешение на доступ к своим данным через механизм авторизации OAuth 2.0.
- Приложение получает access token, который используется для доступа к защищенным ресурсам.
- Токены имеют ограниченный срок действия и могут быть обновлены с помощью refresh token.

Преимущества:

- **Позволяет безопасно предоставлять доступ к ресурсам без передачи паролей.**
- **Поддерживает гранулярный контроль доступа с различными уровнями разрешений (scopes).**
- **Широко принят и стандартизирован, поддерживается многими крупными сервисами (Google, Facebook, GitHub и т.д.).**
- **Поддерживает механизмы обновления и отзыва токенов.**

Недостатки:

- **Сложнее в реализации и настройке по сравнению с другими схемами.**
- **Требует дополнительной инфраструктуры и компонентов (например, сервера авторизации).**
- **Может быть избыточным для простых приложений.**

Схема аутентификации	Описание	Преимущества	Недостатки	Когда использовать
TokenAuthentication	Использует уникальные токены для идентификации пользователей.	Простота, подходит для мобильных приложений, легко обновляемые токены.	Токены хранятся на сервере, отсутствие встроенного механизма обновления, риск компрометации токенов.	Малые проекты, внутренние API, простота и скорость реализации.
SessionAuthentication	Использует стандартный механизм сессий Django для аутентификации.	Глубокая интеграция с Django, поддержка CSRF-защиты, удобна для веб-приложений.	Неудобна для мобильных приложений и RESTful API, сложность масштабирования	Традиционные веб-приложения, взаимодействие с браузерами.
BasicAuthentication	Реализует базовую HTTP-аутентификацию с передачей учетных данных в заголовке.	Простота, поддерживается большинством HTTP-клиентов.	Передача данных без шифрования (без HTTPS), риск перехвата данных, неудобство управления сеансами.	Тестирование, контролируемые среды.
JWTAuthentication	Использует JSON Web Tokens для безопасной и статусной аутентификации.	Статусность, масштабируемость, возможность обновления токенов, поддержка многими библиотеками.	Сложность отзыва токенов, необходимость настройки безопасности, риск компрометации токенов.	Большие распределенные системы, мобильные приложения, SPA.
OAuth2Authentication	Реализует OAuth 2.0 для предоставления ограниченного доступа к ресурсам пользователя.	Безопасное предоставление доступа, поддержка различных уровней разрешений, стандартность	Сложность в реализации, необходимость дополнительной инфраструктуры, может быть избыточным для простых приложений.	Интеграция с внешними сервисами, высокая степень безопасности.

JWT

JWT = **J**SON **W**eb **T**oken – стандарт токенов для JSON.

Сплошные плюсы:

1. Не требует хранения состояния на сервере
2. Может содержать любые данные в полезной нагрузке
3. 2 в 1: безопасность и опенсорс.

Структура:

HEADER

PAYLOAD

SIGNATURE

Пример токена

HEADER

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.

Алгоритм шифрования: "HS256" (HMAC с использованием SHA-256)

PAYLOAD

eyJ1c2VybmFtZSI6ImFkbWludWliaWF0IjoxNjI1NTQ3NzU2LCJleHAiOiE2MjE1NDg2NTYsIm9yaWdfaWF0IjoxNjI1NTQ3NzU2fQ.

Утверждения (claims):

"username": "admin" (имя пользователя)

"iat": 1621547756 (время выдачи токена)

"exp": 1621548656 (время истечения токена)

"orig_iat": 1621547756 (исходное время выдачи токена)

SIGNATURE

WUxT5mLuUV2VZJiH4ibnVW0s6PZroFV68d1-07kmIEU

Заголовок + нагрузка + ключ сервера = подпись в токене

Настройка аутентификации

Модуль `django-rest-framework-simplejwt` 

`settings.py`

```
INSTALLED_APPS = [  
# другие приложения  
    'rest_framework',  
    'rest_framework_simplejwt',  
]  
REST_FRAMEWORK = {  
    'DEFAULT_PERMISSION_CLASSES': [  
        'rest_framework.permissions.IsAuthenticated',  
    ],  
    'DEFAULT_AUTHENTICATION_CLASSES': [  
        'rest_framework_simplejwt.authentication.JWTAuthentication',  
    ],  
}
```

Аутентификация через токен

Используется модуль **django-rest-framework-simplejwt**.

JWT-токен передается в заголовке запроса (обычно с префиксом "Bearer") или в виде параметра запроса для аутентификации и авторизации пользователей в Django REST API.

urls.py

```
from rest_framework_simplejwt.views import TokenObtainPairView,
TokenRefreshView

urlpatterns = [
    # URL для получения токена аутентификации
    path('api/token/', TokenObtainPairView.as_view(),
name='token_obtain_pair'),

    # URL для обновления токена аутентификации
    path('api/token/refresh/', TokenRefreshView.as_view(),
name='token_refresh'),
]
```

Добавление аутентификации через токен

```
from datetime import timedelta
```

```
# Настройка срока действия токена
```

```
SIMPLE_JWT = {  
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=60) ,  
    'REFRESH_TOKEN_LIFETIME': timedelta(days=1) ,  
}
```

Authentication_classes, permission_classes.

permission_classes — это список классов, определяющих, какие права доступа требуются для выполнения запроса.

IsAuthenticated: Разрешает доступ только аутентифицированным пользователям

IsAdminUser: Доступ предоставляется только администраторам (пользователям с флагом `is_staff`).

IsAuthenticatedOrReadOnly: Дает полный доступ аутентифицированным пользователям, а неаутентифицированные пользователи могут только просматривать данные (GET-запросы).

AllowAny: Разрешает доступ всем пользователям, независимо от их статуса аутентификации. Это значение по умолчанию.

authentication_classes в Django REST Framework определяет классы аутентификации, которые будут использоваться для проверки подлинности пользователя в определенном представлении (view).

Настройка представлений (views.py)

Те представления, которые требуют аутентификацию, должны содержать поля **authentication_classes** и **permission_classes**.

Пример:

```
from rest_framework.views import APIView
from rest_framework.permissions import IsAuthenticated
from rest_framework_simplejwt.authentication import JWTAuthentication
```

```
class MySecureView(APIView):
    authentication_classes = [JWTAuthentication]
    permission_classes = [IsAuthenticated]

    def get(self, request):
        return Response(data={"message": "Это защищенное представление!"})
```


Headers

Заголовок (header) — это дополнительная информация, которая передается вместе с запросом или ответом между клиентом и сервером.

```
headers = {  
    "Authorization": f"Bearer {token}"  
}  
protected_url = "http://127.0.0.1:8000/api/"  
protected_response = requests.get(protected_url, headers=headers)
```

Практика

1. Настройте JWT-аутентификацию в проекте CarBrands.
2. Создайте новый эндпойнт `/api/token/`, который будет отвечать за выдачу JWT токена при успешной аутентификации пользователя.
3. Защитите эндпойнты `/car_brands/` и `/car_brands/<id>/` с использованием JWT аутентификации. Только аутентифицированные пользователи должны иметь доступ к этим эндпоинтам.